



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Representación contextual

Atención y Transformers, sin matemática pesada
Módulo de Análisis de Datos de Texto · Clase 4 de 5

¿Dónde quedamos y qué nos faltaba?

Los embeddings de la clase 3 dan **un solo vector por palabra**, fijo, sin importar la frase en que aparezca.

Lo que necesitamos: que el vector de “banco” sea distinto en “me senté en el banco del parque” y en “fui al banco a sacar dinero”. La representación debe depender del contexto.

La misma palabra debería tener distinta representación según las palabras que la rodean.

El mecanismo de atención

La idea de atención

Intuición

Cada palabra de la frase “mira” a todas las demás y decide a cuáles prestar atención. Luego se reexpresa combinando la información de las palabras relevantes para ella.

Ejemplo: para entender “banco”, la palabra mira a “parque” o a “dinero” en la misma frase. Según cuál esté presente, su representación cambia. Así nace una representación contextual.

Atención en tres pasos (sin fórmulas)

1. Comparar

Cada palabra mide cuánto se relaciona con cada otra palabra de la frase.

2. Ponderar

Esas relaciones se convierten en pesos que suman 1: a quién prestar más o menos atención.

3. Combinar

Cada palabra se vuelve un promedio ponderado de las demás: su versión contextual.

Bajo el capó hay productos punto (la misma idea de similitud de la clase 2) y un softmax que reparte los pesos como fracciones que suman 1. **Pero la intuición es lo que importa: mirar, ponderar, combinar.**

La arquitectura Transformer

A vuelo de pájaro

Tokens → Embeddings + posición → Bloques de atención → Representación contextual

No deducimos cada componente: basta entender el flujo. El texto entra como tokens, se convierte en vectores, pasa por capas de atención que lo contextualizan, y sale una representación que “entiende” cada palabra en su contexto.

El paper fundacional: ***“Attention is All You Need” (2017)***. Es la base de BERT, GPT y todos los modelos modernos.

Dos familias de modelos

Encoder — “entender”

BERT y similares.

Leen toda la frase de una vez.

Útiles para clasificar, buscar, analizar sentimiento.

Decoder — “generar”

GPT y similares.

Predicen la siguiente palabra.

Útiles para generar texto.

Esta distinción es la clave para la próxima clase: si un modelo predice la siguiente palabra, es un generador.

Laboratorio · Clase 4

Notebook — Atención (demostrativo)

Como no usamos librerías de deep learning, el laboratorio es ilustrativo y conceptual.

- Implementar en NumPy/Python un mini-cálculo de atención sobre 3–4 vectores de juguete: comparar, aplicar softmax, combinar.
- Visualizar la matriz de pesos de atención (quién mira a quién) como un heatmap, reusando lo aprendido en la clase 2.
- Ejercicio: cambiar el contexto y observar cómo cambian los pesos y la representación resultante.

Stack: Python · NumPy · matplotlib